

National University of Computer & Emerging Sciences — Islamabad Campus		
Object-Oriented Programming	Spring 2026	Project
Deadline: 10th May 2026 11:59 pm		Total Marks: TBD

General Instructions

1. **Start early. The design complexity of this project requires careful planning before any code is written.**
2. This assignment is in groups of 3 (pairs can be formed in special cases, notify both TAs and instructor). The code you write must reflect your own understanding of every design decision made.
3. You are encouraged to seek conceptual help from your professor or teaching assistants via Google Classroom, but implementation must be entirely your own.
4. Marks are allocated for good programming practices, including consistent indentation, meaningful comments that explain design rationale, and proper management of dynamically allocated memory.
5. You may include your name, section, roll number, instructor, and TA at the top of your main file for the possibility of bonus marks.
6. Marking will be conducted through a viva and a code review session. You must be able to justify every class, relationship, and design decision.
7. Your code must compile and execute without errors. Non-compiling code will not be entertained.
8. Modification of code after submission will not be tolerated.
9. Use of built-in container libraries (such as `<vector>`, `<list>`, `<map>`, `<string>` etc.) is strictly prohibited. All dynamic collections must be managed manually.
10. You may use built-in library for emailing.
11. Detailed inline comments explaining your logic are required, particularly for memory management, recursive operations, and relationship modelling.
12. **Plagiarism of any form will result in zero marks.**
13. **Implement a different class in each file, one class per .cpp file, .h files are optional. Give the file the same name as the class.**
14. **Submission: Compress a folder named "Group_Number.zip" containing your .cpp and .h files as well as a UML in pdf format (Group_Number.pdf). Only .zip submissions via Google Classroom are accepted.**

Note: Follow the given instructions to the letter. Failure to do so may result in zero marks.

1. Introduction

Fatima and Musa are two students in the Cyber Security batch of 2025 who have been passionate about competitive programming since their first day. They regularly practise on platforms like LeetCode and Codeforces, grinding through problems late into the night and occasionally dragging each other into contests neither of them feel quite ready for. For their OOP semester project, they are tasked with building something they already know from the other side of the screen, an Online Judge platform of their own.

The platform must be built entirely in C++, using all four pillars of object-oriented programming, without standard library containers, without external databases, and without any kernel-level tooling. Earlier in the semester, both Fatima and Musa built a custom Virtual File System for Assignment 3. This project builds directly on top of that work, the VFS is the storage backbone for everything the judge platform needs to persist.

1.1 Project Summary

Team	3 members (2 in special cases only)
Language	C++
Storage	Custom VFS from Assignment 3
UI	SFML OpenGL HTML/CSS/JS
No STL	Standard containers are banned throughout
No <code>std::string</code>	All text is handled via character arrays
No External DBs	All persistence goes through the VFS

2. Technical Constraints

Before reading the scenario, understand what the platform is not allowed to use. These constraints are non-negotiable and apply to every part of the codebase.

Constraint	What it means
No Standard Library Containers	You may not use built-in containers from the C++ standard library. Any list, collection, map, or queue needed by the system must be designed and implemented by your team from scratch.
No <code>std::string</code>	Every piece of text in the system, usernames, problem statements, code buffers, file paths, must be stored and manipulated as a raw character array. String operations must be handled manually or through custom wrappers.
No Kernel-Level Sandboxing	You are not permitted to use low-level system calls to create isolated processes, set resource limits, or sandbox user submissions. Evaluation must use a simpler approach available to any standard C++ program.
No External Databases	There is no SQLite, no file-based database, no third-party storage layer. Everything that must be saved between sessions must go through the VFS your team already built.
Memory Discipline	Every dynamic allocation must be properly freed. Destructors must be

	thoughtfully written. The system will be assessed for memory correctness.
--	---

3. The Scenario

The following narrative describes two students, Fatima and Musa, as they interact with an online judge platform. Read it carefully. Every expectation they express, every friction they encounter, and every feature they take for granted is a requirement waiting to be discovered.

3.1 First Contact

Fatima hears about a competitive programming platform being introduced for the university's CS programme. She opens it for the first time on her laptop. The platform greets her with a clean, minimal interface, no clutter, no overwhelming menus. She sees options to register or log in.

She registers using a username, email address, and password. The system tells her to verify her account before she can proceed. Shortly after, she receives what appears to be a confirmation link or token. She follows it, and her account becomes active. She notices that the platform has already organised a space for her behind the scenes, somewhere to store her work, her progress, and eventually her competition results.

She spends a few minutes exploring. She sees a library of problems, clearly grouped by difficulty. Some problems look approachable; others seem designed to challenge experienced programmers. She opens a few and reads through the problem statements and constraints. Each one includes a handful of example inputs and expected outputs that she can use to test her thinking.

3.2 Musa Joins

Musa registers the next day. He goes through the same process as Fatima and notices that after signing up, the platform appears to have a contest. The contest has a different selection of problems than what Fatima described. Fatima mentioned she had a relatively balanced set, while Musa's leans harder. The platform clearly supports multiple contest configurations, each with its own mix of problem difficulties, with the users being able to choose between difficulty 1-5.

NOTE The grouping mechanism (of questions) and what differentiates each contest is a design decision left to you. Consider what makes one contest harder or easier than another, and how you would represent that distinction in code.

3.3 Starting the Contest

Fatima decides to start her contest. She clicks a button and immediately notices a timer appear on screen. The clock is counting down. She feels the pressure. She opens her first problem and begins reading.

The platform provides her with a coding area — a clean editor where she can write her solution. She starts typing. She submits her first attempt and receives a verdict almost immediately. It tells her whether her solution passed all the test cases or not. When it fails, she gets just enough information to understand what went wrong, but not the answer itself.

She moves to a second problem. She notices that wrong submissions seem to affect her eventual score, not just whether she solved the problem, but how quickly and how cleanly. The scoring feels designed to reward accuracy and speed together.

3.4 Musa Leaves Midway

Musa starts his contest but gets pulled away unexpectedly. He has written about half of a solution for one of the problems but has not yet submitted it. He saves his progress and closes the platform entirely.

When he returns later, he logs back in and finds that the platform remembers exactly where he left off. His partial solution is still there, waiting for him. He picks up from where he stopped, finishes the solution, and submits it. The platform continues counting time from when the contest started, his absence did not pause the clock.

NOTE The ability for a user to save work mid-session and return to it later is a core persistence requirement. Think about what information needs to survive a complete shutdown of the program, and how that information should be stored using your VFS. Both for competition and normal questions. (Normal question code solutions should live to the end of the users account)

3.5 The Contest Ends

Fatima's timer reaches zero. The platform automatically ends the contest and shows her a results screen. She sees a breakdown of which problems she solved, how many attempts she made, and how her time affected her final score. The score calculation feels fair, someone who solved three problems quickly scores higher than someone who also solved three but spent more time and made more mistakes.

She is then prompted with a choice: keep the code she wrote during the contest or discard it. She decides to keep it. The platform organises it neatly, by contest name and date, so she can refer to it later.

Musa, who ran out of time without finishing everything, sees a similar screen. His incomplete submissions are noted. He chooses to delete his contest files; he does not want them kept.

3.6 Later — Account Management

A few weeks later, Fatima decides to delete her account entirely. She goes through a confirmation process. Once confirmed, the platform removes every trace of her data, her profile, her saved code, her contest history, and her progress file. Nothing of hers remains on the system.

Musa continues using the platform. He logs in and out across multiple sessions, always finding his history intact and his interface consistent.

4. What the Platform Must Support

From Fatima and Musa's experience, the following expectations emerge naturally. These are not a requirements checklist; they are observations drawn from the scenario. How you model and implement each one is entirely your decision.

4.1 User Management

The platform must support the full lifecycle of a user: registration, identity verification, login, active session, and account deletion. When a user is created, the system should prepare a structured space for them within the VFS — a dedicated area for their profile, their progress, and their contest records. When a user is deleted, that space and everything inside it must be completely removed.

Sessions must be properly managed. A user who is not logged in should not be able to access contest or problem functionality.

4.2 Problem Bank

The platform hosts a library of problems across multiple difficulty tiers. Each problem has a full description, constraints, and a set of test cases that define correct output for given inputs. The number and distribution of problems across difficulty levels is a design decision you must make (10 questions per difficulty min) and the bank must be large enough to support multiple contest groups with different mixes.

Problems and test cases must be stored in the VFS and loaded into memory at runtime. They are read-only from the user's perspective — users interact with them, but do not modify them.

4.3 Contest System

The platform supports multiple contest variants. Each variant presents a different selection of problems with a different difficulty composition. Users choose one variant. Within their variant, the specific problems shown may be randomised from the available pool each time a contest runs, but the difficulty distribution remains consistent with the contests's profile.

A contest is time-bound. Once started, a countdown begins. The contest ends either when time expires or when the user chooses to end it manually. Partial work can be saved at any point. When the contest concludes, the user must be presented with a post-contest decision: retain the generated code in a structured location within the VFS, or discard it entirely.

4.4 Scoring

Scoring should reflect both correctness and efficiency. A user who solves a problem correctly on the first attempt and quickly should score more than one who solves the same problem after multiple failed submissions and with little time remaining. The exact formula is your design decision — but it should be meaningful, transparent to the user, and consistent across all groups.

4.5 Code Submission & Evaluation

When a user submits a solution, the system evaluates it against the problem's test cases and returns a verdict. The evaluation must happen without spawning sandboxed subprocesses or using any kernel-level isolation. The result should clearly communicate whether the submission passed, and if not, give enough feedback to be useful.

HINT HINT — Evaluation approach: Consider wrapping the user's submitted code inside a known template structure, compiling it using a standard system call to the compiler, and then executing the resulting binary while redirecting standard input from a test case file and capturing standard output to a separate file. That output file can then be opened and compared line by line against the expected output. No sandboxing. No special libraries. Just compilation, execution, and comparison.

4.6 Persistence & VFS

All data that must survive a program shutdown — user profiles, problem bank, in-progress solutions, contest records — must be stored through the VFS your team built in Assignment 3. You do not need redesign the VFS; you should design the online judge to sit cleanly on top of it. Think carefully about how to organise data within the VFS so that it is easy to retrieve, update, and delete in a structured way.

IMPORTANT Do not add external file I/O outside the VFS for application data. Temporary files created during evaluation (such as compiled binaries or output captures) are the only exception, and even those should be cleaned up immediately after use.

5. Design Guidance

This section does not tell you what to build. It gives you principles to think with as you design the system.

5.1 Architectural Thinking

The system should be designed in layers. There is a layer that handles the user interface and input. There is a layer that contains the business logic — contest management, scoring, evaluation. There is a layer that deals with domain objects like users and problems. And there is the VFS layer that handles all persistence. These layers should be loosely coupled: changing the UI should not require rewriting the scoring logic.

Within the business logic layer, certain components share common behaviour while behaving differently in specific ways. A contest group, for instance, has a concept of building a question set — but how it builds that set differs by group. The system should be designed so that these shared behaviours are captured in a common contract, and individual variants implement that contract in their own way. This is a direct invitation to use polymorphism thoughtfully.

Similarly, any object that needs to be saved and loaded from the VFS has a shared responsibility. The mechanism for doing that should be consistent and reusable across different types of objects, not reimplemented every time.

5.2 Encapsulation & Abstraction

The internal state of objects — user credentials, problem data, contest configuration — should not be directly accessible from outside those objects. Design your classes so that what they expose is intentional, and what they hide is protected. A well-designed class should be usable without needing to know how it works internally.

At the same time, avoid over-exposing complexity. When one component needs to interact with another, it should do so through a clean, minimal interface. The fewer assumptions each component makes about others, the more maintainable the system will be.

5.3 Custom Data Structures

Because standard library containers are not available, your team will need to implement the data structures required by the system. These might include dynamic arrays, linked structures, or associative lookups. Design these as general-purpose tools — they should not be tightly coupled to specific application logic. A well-implemented collection is reusable anywhere in the system.

Similarly, because `std::string` is unavailable, consider building a character array wrapper that encapsulates common string operations in a safe and consistent way. This will prevent repetitive low-level code scattered throughout the codebase.

5.4 UI Design

The user interface is a significant part of the experience. Fatima and Musa's scenario implies several screens: a login and registration view, a dashboard, a problem browser, a coding environment with a visible timer, a results screen, and a post-contest decision prompt. Each of these must be thoughtfully designed.

Regardless of which UI framework you choose — SFML, OpenGL with a widget layer, or an HTML/CSS/JS interface — the interface must be clean, consistent, and responsive. A platform that judges code should look like it was built with care. Consider visual hierarchy, feedback states (what happens when an error occurs, when a submission passes, when time is running low), and a coherent colour scheme. The timer should communicate urgency as it counts down.

IMPORTANT UI quality will be assessed. A functional system with a poor interface will score less than one that is both functional and polished. Invest time in design — not just in functionality.

6. Object-Oriented Design Requirements

The system must demonstrate all four pillars of object-oriented programming. This is not optional — it is a graded criterion. Below is a summary of what each pillar must look like in the context of this project.

Pillar	What it means for this project
Encapsulation	Internal data — passwords, file paths, question content — must be hidden behind well-designed interfaces. No public data members. Access is controlled through methods.
Abstraction	Define contracts for shared behaviours. Anything that needs to be saved to the VFS should share a common interface. Anything that needs to be evaluated should share a common interface. Users of these abstractions should not need to know what type they are working with.
Inheritance	The contest system, in particular, presents a natural inheritance hierarchy. Different contest groups share a common concept of what a contest is, but differ in their configuration. Model this explicitly through base and derived types.
Polymorphism	The system's controller layer should operate against base types wherever possible. If a component handles a contest, it should not need to know which specific group it is — it should simply call the shared interface and let the correct behaviour run. This must be evident in the design.

7. Implementation Checklist

Use the following checklist to track your progress. Each item represents a graded component of the project. No implementation detail is prescribed here — the design is yours.

Core Architecture

- OOP principles (all four pillars demonstrably applied)
- Layered, modular system design
- Polymorphic contest hierarchy
- Shared persistence contract across domain objects
- Shared evaluation contract for submission handling

Data & Storage

- Custom dynamic array
- Custom linked structure
- Custom associative lookup (e.g. key-value store)
- Character array string wrapper
- VFS integration for all persistent data
- Problem bank seeded into VFS at startup

User System

- Registration with identity verification
- Secure credential handling
- Login and session management
- Dedicated per-user VFS space
- Full account deletion (recursive VFS wipe)

Contest System

- Multiple contest group variants with different difficulty profiles
- Automated group assignment on registration
- Randomised problem selection per group
- Countdown timer with auto-end behaviour
- Mid-contest save and resume
- Post-contest save or discard decision
- Score calculation factoring time and attempts

Evaluation Engine

- Code submission and template injection
 - Compilation via system call
 - Execution against test cases with stream redirection
 - Output comparison and verdict generation
 - Temporary file cleanup after evaluation
-

UI / UX

- Login and registration screen
- Email verification screen
- Dashboard with contest and problem status
- Problem browser with difficulty filtering
- Code editor with live timer
- Verdict and results screen
- Post-contest action prompt
- Consistent visual design and feedback states

Code Quality

- Zero memory leaks (verified with Valgrind or equivalent)
- No STL containers anywhere in the codebase
- No `std::string` anywhere in the codebase
- Consistent naming and formatting conventions
- Clean build with no warnings
- Build system provided (Makefile or CMakeLists.txt)

Happy coding 😊
Online Judge on VFS
